

12장- 네트워크 프로그래밍



Socket



네트워크 기초

네트워크

- 홈 네트워크 : 컴퓨터가 방마다 있고, 이들 컴퓨터를 유.무선 등의 통신회선으로 연결(LAN)
- 지역 네트워크 : 회사, 건물, 특정 영역에 존재하는 컴퓨터를 통신회선으로 연결한 것(MAN)
- 인터넷 : 지역 네트워크를 통신 회선으로 연결한 것(WAN)

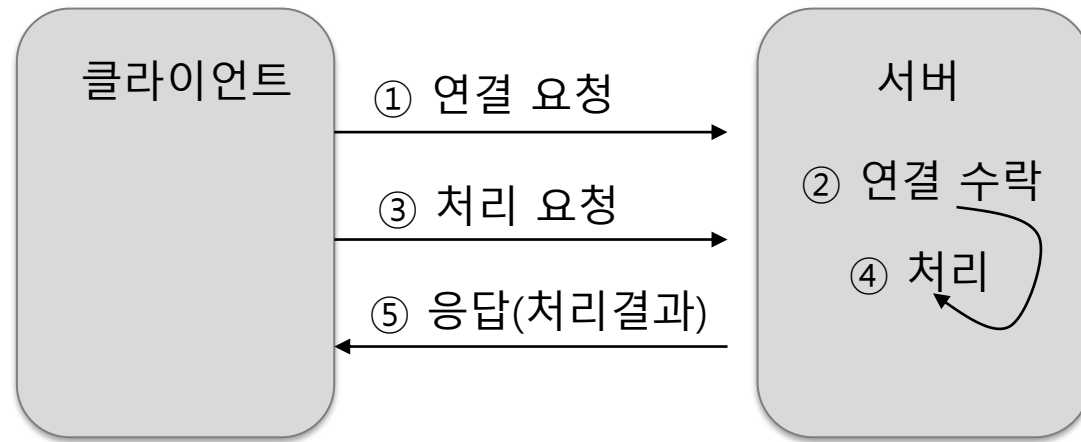
서버와 클라이언트

- **서버(Server)** : 서비스를 제공하는 프로그램
 - 웹서버, FTP서버, DBMS 서버, 메신저 서버
 - 클라이언트의 연결을 수락하고, 요청 내용을 처리한 후 응답을 보내는 역할
- **클라이언트(Client)** : 서비스를 받는 프로그램
 - 웹브라우저, FTP클라이언트, 메신저
 - 네트워크 데이터를 필요로 하는 모든 애플리케이션이 해당(모바일 App포함)



서버 / 클라이언트

서버와 클라이언트



- ▶ 클라이언트/서버(C/S: client/server) : 한 개의 서버와 다수의 클라이언트로 구성
- ▶ P2P(Peer to Peer) : 두 개의 프로그램이 서버인 동시에 클라이언트 역할을 함
먼저 접속을 시동한 컴퓨터가 클라이언트가 된다. (1:1 채팅과, 파일 공유 프로그램)



IP 주소와 포트(Port)

IP 주소

- IP(Internet Protocol) 주소 : 컴퓨터의 고유한 주소 - IPv4
- xxx.xxx.xxx.xxx(xxx는 0~255 사이의 정수)
- 네트워크 어댑터(Lan 카드) 마다 할당 - 유선/무선 랜카드

명령 프롬프트(cmd)

C:W>ipconfig 입력

```
C:\Users\김기웅>ipconfig
```

```
Windows IP 구성
```

```
이더넷 어댑터 이더넷:
```

```
미디어 상태 . . . . . : 미디어 연결 끊김  
연결별 DNS 접미사 . . . . . :
```

```
무선 LAN 어댑터 Wi-Fi:
```

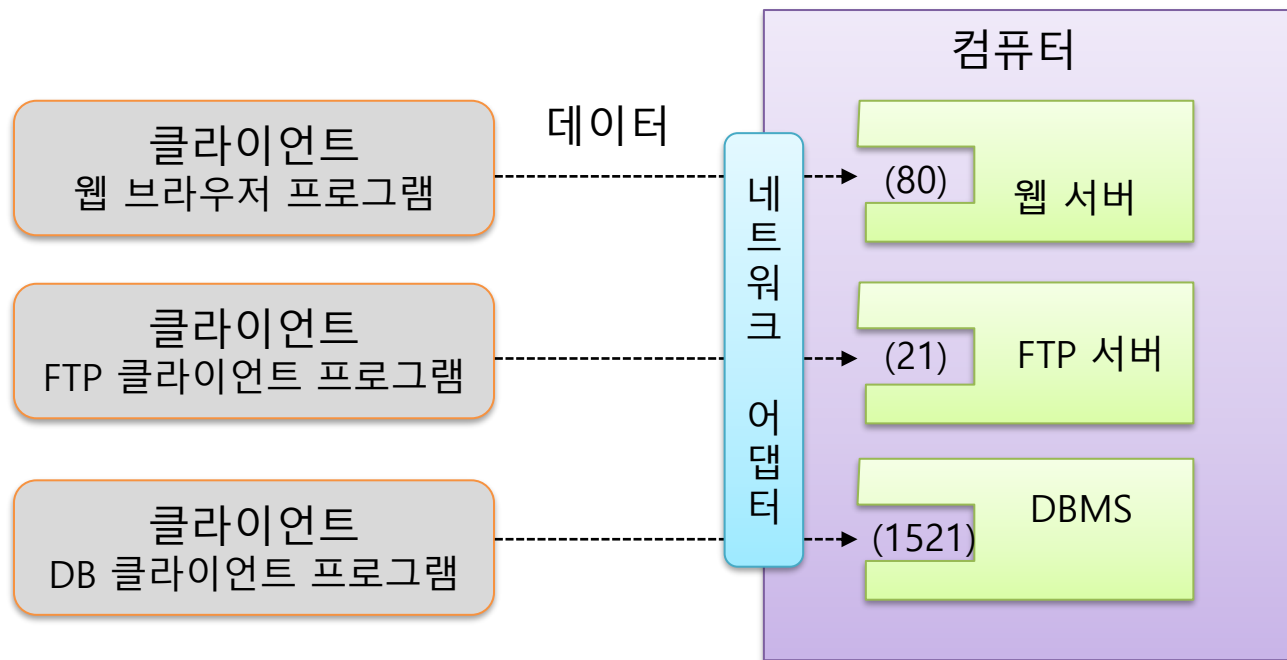
```
연결별 DNS 접미사 . . . . . :  
링크-로컬 IPv6 주소 . . . . . : fe80::fdb4:956c:171d:19c7%14  
IPv4 주소 . . . . . : 192.168.0.6  
서브넷 마스크 . . . . . : 255.255.255.0  
기본 게이트웨이 . . . . . : 192.168.0.1
```



IP 주소와 포트(Port)

포트(Port)

- 같은 컴퓨터 내에서 프로그램을 식별하는 번호
- 클라이언트는 서버 연결 요청시 IP 주소와 Port를 같이 제공
- 포트번호의 전체 범위 : 0 ~ 65535 범위의 값을 가짐



IP 주소와 포트(Port)

포트(Port)

- Port는 운영체제가 관리하는 서버 프로그램의 연결 번호이다. 서버는 시작할 때 Port 번호에 바인딩한다. 예를 들어 서버는 80번, DBMS(MySQL)는 3306번으로 바인딩 할 수 있음.
- 클라이언트도 서버에서 보낸 정보를 받기 위해서는 Port 번호가 필요한데, 서버와 같이 고정적인 Port 번호에 바인딩하는 것이 아니라 운영체제가 자동으로 부여하는 번호를 사용한다.

클래스	범위	용 도
잘 알려진 포트번호 (Well Know Port Numbers)	0~1023	국제인터넷 주소관리기구(CANN)가 특정 애플리케이션용으로 미리 예약한 Port
예약된 포트번호 (Registered Port Numbers)	1024~49151	회사에서 등록해서 사용할 수 있는 Port
동적인 또는 개인 포트번호 (Dynamic Or Private PortNumbers)	49152~65535	운영체제가 부여하는 동적 Port 또는 개인 목적으로 사용할 수 있는 Port



IP 주소와 포트(Port)

IP 주소 얻어 오기

- **Java.net.InetAddress**
- IP 주소를 표현한 클래스
- 로컬 컴퓨터의 IP 주소 뿐만아니라 도메인 이름을 DNS에서 검색한 후 IP 주소를 가져 오는 기능 제공 (예. www.naver.com -> IP 주소)

1. 로컬 컴퓨터에서 얻기

```
InetAddress ia = InetAddress.getLocalHost();
```

2. 도메인 이름으로 얻기

```
InetAddress ia = InetAddress.getByName(String host)
```

```
InetAddress[ ] iaArr = InetAddress.getAllByName(String host)
```



IP 주소와 포트(Port)

IP 주소 얻어 오기

```
package inetaddress;
import java.net.InetAddress;
import java.net.UnknownHostException;

public class InetAddressEx {

    public static void main(String[] args) {

        try {
            //내 컴퓨터
            InetAddress local = InetAddress.getLocalHost();
            System.out.println("내 컴퓨터 IP 주소 : " + local.getHostAddress());

            //서버 컴퓨터
            //InetAddress server = InetAddress.getByName("www.naver.com");
            //System.out.println(server);
            InetAddress[] servers = InetAddress.getAllByName("www.naver.com");
            for(InetAddress remote : servers)
                System.out.println(remote);
        } catch (UnknownHostException e) {
            e.printStackTrace();
        }
    }
}
```

내 컴퓨터 IP 주소 : 192.168.0.6

네이버 컴퓨터 IP 주소 : www.naver.com/223.130.195.200

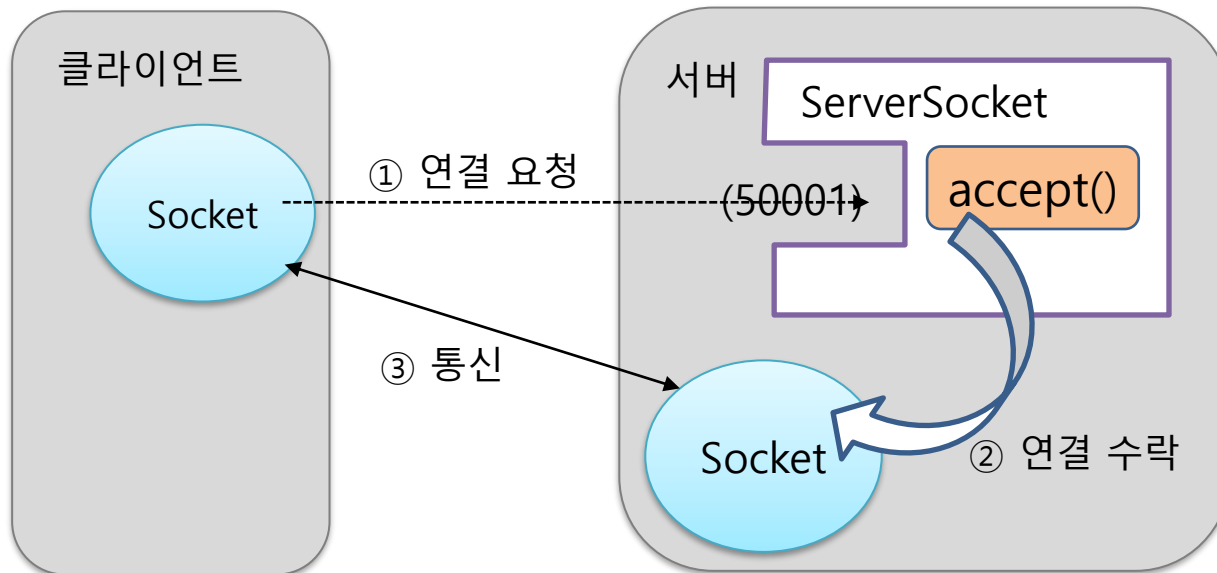
네이버 컴퓨터 IP 주소 : www.naver.com/223.130.195.95



TCP 네트워크

TCP(Transmission Control Protocol)

- 연결 지향적 프로토콜 : 클라이언트와 서버가 연결된 상태에서 데이터를 주고 받는 프로토콜이다.
- 데이터를 정확하고 안정적으로 전달 - 데이터를 순차적으로 보내고 받을 때도 순차적으로 받음
- **Java.net API – ServerSocket 클래스, Socket 클래스**



TCP 네트워킹

```
Problems @ Javadoc Declaration Console x
ServerSample [Java Application] C:\Users\wkiyon\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.f
=====
서버를 종료하려면 q 또는 Q를 입력하고 Enter를 누르세요.
=====
[서버] 시작됨

[서버] 연결 요청을 기다림

[서버] 127.0.0.1의 연결 요청을 수락함
[서버] 127.0.0.1의 연결을 끊음

Console x
<terminated> ClientExample [Java Application] C:\Users\wkiyon\p...
[클라이언트] 연결 성공
[클라이언트] 연결 끊음
```



ServerSocket 생성과 연결 수락

ServerSocket 생성과 연결 수락 – 서버 프로그램

1. ServerSocket 생성과 포트 바인딩

```
ServerSocket serverSocket = new ServerSocket(50001);  
serverSocket.bind(new InetSocketAddress("localhost", 50001);
```

2. 연결 수락

```
try{  
    Socket socket = serverSocket.accept();  
}catch(Exception e){ }
```

3. 연결된 클라이언트 IP 주소 얻기

```
InetSocketAddress socketAddress =  
    (InetSocketAddress)socket.getRemoteSocketAddress();
```

2. 연결 끊기

```
try{  
    serverSocket.close();  
}catch(Exception e){ }
```



ServerSocket 연결 수락

연결 수락 - 서버 프로그램

```
public class ServerSample {
    //서버 소켓 객체 선언
    private static ServerSocket serverSocket;
    //main 스레드 - 키보드 입력 : 서버 종료하는 작업
    //작업 스레드 - 클라이언트 요청 받아서 수락하는 작업
    public static void main(String[] args) {
        System.out.println("=====");
        System.out.println("서버를 종료하려면 q 또는 Q를 입력하고 Enter를 누르세요.");
        System.out.println("=====");

        //TCP 서버 시작
        startServer();

        Scanner scanner = new Scanner(System.in);
        while(true) {
            String key = scanner.nextLine();
            if(key.toLowerCase().equals("q")) {
                break;
            }
        }
        scanner.close();
        //TCP 서버 종료
        stopServer();
    }
}
```



ServerSocket 연결 수락

```
public static void startServer() {  
    //작업 스레드 정의  
    Thread thread = new Thread() {  
        @Override  
        public void run() {  
            //ServerSocket 생성 및 Port 바인딩  
            try {  
                serverSocket = new ServerSocket(50001);  
                System.out.println("[서버] 시작됨"); //실행 : BindException 확인  
  
                while(true) { //여러 클라이언트의 연결 요청 수락을 위해 필요함  
                    System.out.println("\n[서버] 연결 요청을 기다림\n");  
                    //연결 수락  
                    Socket socket = serverSocket.accept();  
  
                    InetAddress isa =  
                        (InetAddress)socket.getRemoteSocketAddress();  
                    //String clientIp = isa.getHostName(); //컴퓨터 이름이 나올수 있음  
                    String clientIp = isa.getHostString();  
                    System.out.println("[서버] " + clientIp + "의 연결 요청을 수락함");  
                    //웹 브라우저에 ip주소:50001 입력 -> 콘솔에 IP 주소 출력됨
```

[서버] 연결 요청을 기다림

[서버] 192.168.35.183의 연결 요청을 수락함

[서버] 192.168.35.183의 연결을 끊음

[서버] 연결 요청을 기다림



ServerSocket 연결 수락

```
//연결 끊기
socket.close();
System.out.println("[서버] " + clientIp + "의 연결을 끊음");
    }
} catch (IOException e) {
    //System.out.println("[서버] " + e.getMessage());
    System.out.println("[서버] " + e.toString()); //한 번 더 실행
}
}
};
thread.start();
}

public static void stopServer() {
    try {
        serverSocket.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

* q를 입력하면 SocketException 발생함

```
q
[서버] 종료됨
[서버] Socket closed
java.net.SocketException: Socket closed
    at java.base/sun.nio.ch.NioSocketImpl.endAccept(NioSocketImpl.java:689)
    at java.base/sun.nio.ch.NioSocketImpl.accept(NioSocketImpl.java:762)
    at java.base/java.net.ServerSocket.implAccept(ServerSocket.java:675)
    at java.base/java.net.ServerSocket.platformImplAccept(ServerSocket.java:641)
    at java.base/java.net.ServerSocket.implAccept(ServerSocket.java:617)
    at java.base/java.net.ServerSocket.implAccept(ServerSocket.java:574)
    at java.base/java.net.ServerSocket.accept(ServerSocket.java:532)
    at server.ServerSample$1.run(ServerSample.java:47)
```



TCP 클라이언트

Socket 생성과 연결 요청 – 소켓 클라이언트

1. Socket 생성과 포트 바인딩 -> 서버에 연결 요청

```
Socket socket = new Socket("localhost", 50001);
```

2. 연결 끊기

```
socket.close();
```



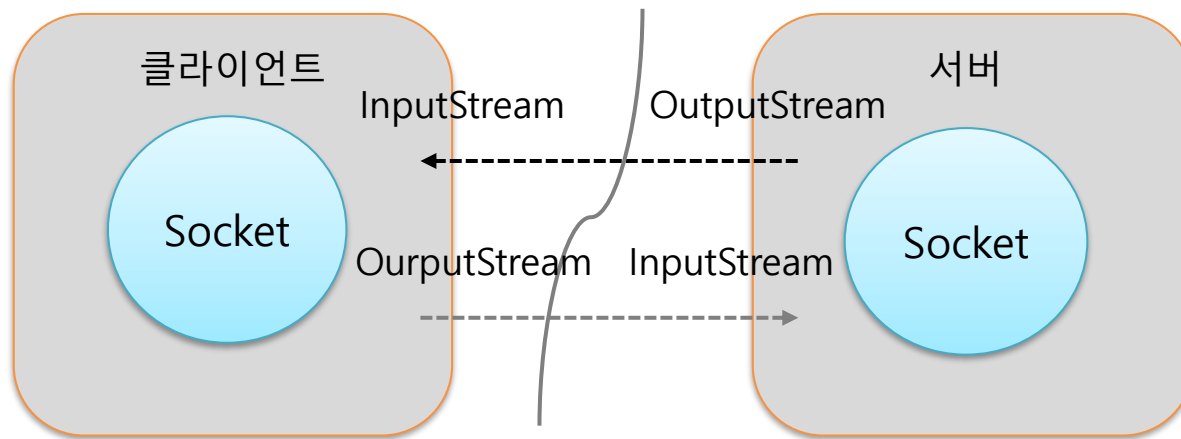
Socket 연결 요청

```
public class ClientExample {  
  
    public static void main(String[] args) {  
        try {  
            //Socket 생성과 동시에 연결 요청  
            Socket socket = new Socket("localhost", 50001);  
  
            System.out.println("[클라이언트] 연결 성공");  
  
            socket.close();  
            System.out.println("[클라이언트] 연결 끊음");  
        } catch (UnknownHostException e) {  
            //IP 또는 도메인 표기 방법이 잘못된 경우  
            System.out.println("UnknownHostException" + e.toString());  
        } catch (IOException e) {  
            //IP 또는 포트번호가 잘못된 경우  
            System.out.println("IOException" + e.toString());  
        }  
    }  
}
```



Socket 데이터 보내고 받기

Socket 데이터 통신



//입력스트림 얻기

InputStream is = socket.getInputStream()

//출력스트림 얻기

OutputStream os = socket.getOutputStream()



Socket 데이터 보내고 받기

Socket 데이터 통신

➤ 데이터 보내기(쓰기)

```
String data = "보낼 데이터";  
byte[ ] bytes = data.getBytes("UTF-8");  
OutputStream os = socket.getOutputStream()  
os.write(bytes);  
os.flush()
```

➤ 데이터 받기(읽기)

```
byte[ ] bytes = new byte[100];  
InputStream inputStream = socket.getInputStream()  
int readByteCount = inputStream.read(bytes);  
String data = new String(bytes, 0, readByteCount, "UTF-8")
```



Socket 데이터 보내고 받기

[클라이언트] 연결 성공
[클라이언트] 데이터 보냄: 오늘도 즐거운 하루 되세요~
[클라이언트] 데이터 받음: 오늘도 즐거운 하루 되세요~
[클라이언트] 연결 끊음

=====

서버를 종료하려면 q 또는 Q를 입력하고 Enter를 누르세요.

=====

[서버] 시작됨

[서버] 연결 요청을 기다림

[서버] 127.0.0.1의 연결 요청을 수락함

[서버] 받은 데이터를 다시 보냄: 오늘도 즐거운 하루 되세요~

[서버] 127.0.0.1의 연결을 끊음



ServerSocket 데이터 보내고 받기

Socket 데이터 통신 – 서버 프로그램 (EcoServer.java)

```
while(true) {
    System.out.println("\n[서버] 연결 요청을 기다림\n");
    Socket socket = serverSocket.accept();

    InetAddress isa = (InetAddress)socket.getRemoteSocketAddress();
    //String clientIp = isa.getHostName(); //컴퓨터 이름이 나올수 있음
    String clientIp = isa.getHostString();
    System.out.println("[서버] " + clientIp + "의 연결 요청을 수락함");
    //-----
    //데이터 받기
    InputStream is = socket.getInputStream();
    byte[] bytes = new byte[1024]; //1KB
    int readByteCount = is.read(bytes); //읽은 바이트 수
    String message = new String(bytes, 0, readByteCount, "utf-8"); //디코딩 문자셋

    //데이터 보내기
    OutputStream os = socket.getOutputStream();
    bytes = message.getBytes("utf-8"); //인코딩 문자셋
    os.write(bytes);
    os.flush();
    System.out.println("[서버] 받은 데이터를 다시 보냄: " + message);
    //-----
}
```



ServerSocket 데이터 보내고 받기

보조 스트림 사용하기 – DataInputStream, DataOutputStream

```
//데이터 받기
DataInputStream dis = new DataInputStream(socket.getInputStream());
String message = dis.readUTF();

//데이터 보내기
DataOutputStream dos = new DataOutputStream(socket.getOutputStream());
dos.writeUTF(message);
dos.flush();
System.out.println("[서버] 받은 데이터를 다시 보냄: " + message);
//-----
```



Socket 데이터 보내고 받기

Socket 데이터 통신 – 클라이언트 프로그램(EcoClient.java)

```
try {
    Socket socket = new Socket("localhost", 50001);

    System.out.println("[클라이언트] 연결 성공");
    //-----
    //데이터 보내기
    String sendMessage = "오늘도 즐거운 하루 되세요~";
    OutputStream os = socket.getOutputStream();
    byte[] bytes = sendMessage.getBytes("utf-8");
    os.write(bytes);
    os.flush();
    System.out.println("[클라이언트] 데이터 보냄: " + sendMessage);

    //데이터 받기
    InputStream is = socket.getInputStream();
    bytes = new byte[1024];
    int readByteCount = is.read(bytes); //읽은 바이트 수
    //디코딩 문자셋
    String receiveMessage = new String(bytes, 0, readByteCount, "utf-8");
    System.out.println("[클라이언트] 데이터 받음: " + receiveMessage);
    //-----
}
```



Socket 데이터 보내고 받기

보조 스트림 사용하기 – DataInputStream, DataOutputStream

```
//데이터 보내기
String sendMessage = "오늘도 즐거운 하루 되세요~";
DataOutputStream dos = new DataOutputStream(socket.getOutputStream());
dos.writeUTF(sendMessage);
dos.flush();
System.out.println("[클라이언트] 데이터 보냄: " + sendMessage);

//데이터 받기
DataInputStream dis = new DataInputStream(socket.getInputStream());
String receiveMessage = dis.readUTF();
System.out.println("[클라이언트] 데이터 받음: " + receiveMessage);
//-----
```

